Temperature Monitoring and Alert System Report

Team: Group1

David Chang , Azin Shahrokhi , Ione Axelrod

1.Introduction

Many laboratories lack automated environmental monitoring, creating safety risks for equipment and personnel. Our system provides real-time Fahrenheit tracking with a multi-layered alert interface: a 16x2 LCD for data, an RGB LED for status (Blue/Green/Red), and a 1500Hz audible alarm triggered at 80°F. The project began as a Tinkercad simulation using a TMP36 sensor to validate logic and pin assignments before moving to physical hardware.

2. System Design Overview

The system operates on a "Read-Process-Update" cycle. Every second, the Arduino polls the sensors, applies a manual calibration "nudge," and updates all outputs to reflect the current safety state.

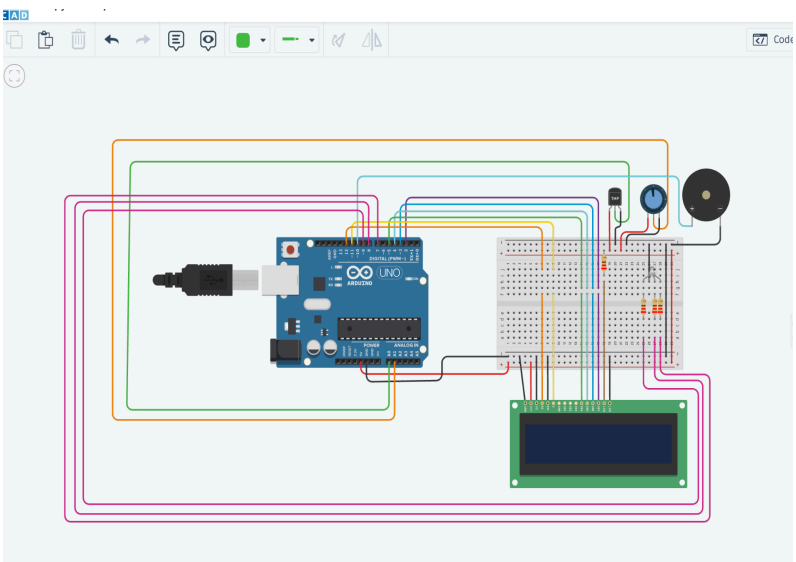
- **Input Subsystem:** DHT11 digital sensor for temperature and a 10k potentiometer for ±10°F manual calibration.
- **Processing Subsystem:** An Elegoo Mega R3 runs C++ logic to evaluate safety thresholds.
- **Output Subsystem:** LCD1602 for numerical data, RGB LED for visual status, and a passive buzzer for audible alarms.

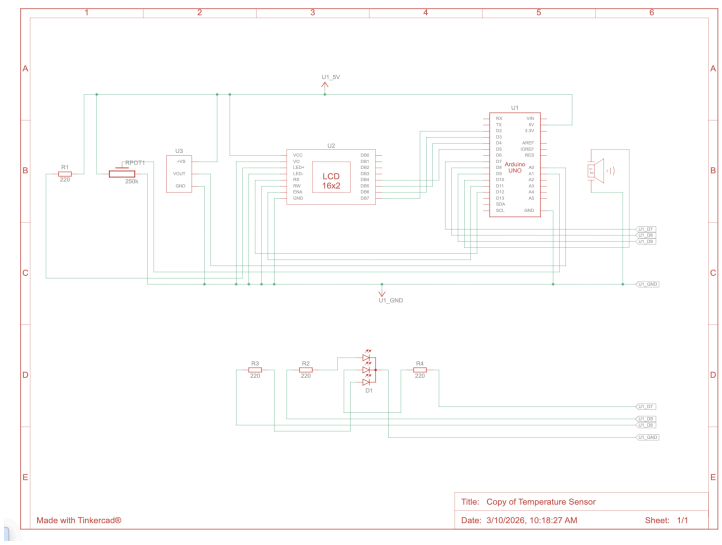
3. Tinkercad Implementation Summary

We used Tinkercad to verify code stability and pin mapping before the physical build. By adjusting virtual sliders, we confirmed the "Too Hot!!" alert and buzzer triggered exactly at 80°F.

- **Parts List:** Arduino Uno, TMP36 sensor, LCD1602, RGB LED, Piezo buzzer, and resistors .
- **Simulation Testing:** Verified logic for LCD (Pins 2-5,11,12), RGB (Pins7-9), and Buzzer (Pin 10).

Circuit View & Schematic View:





We verified the system logic by adjusting virtual temperature sliders and potentiometers to trigger simulated heat spikes. The simulation successfully confirmed that the "Too Hot!!" alert and buzzer activated at the 80°F threshold, proving our pin assignments were ready for physical implementation. This testing phase concluded with a plan to use a physical ice bag to validate the system's transition to the "Cold" (Blue) state.

Simulated Parts List

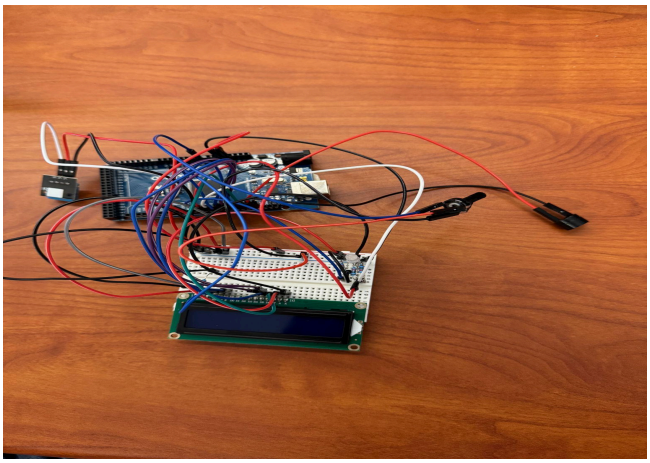
- **Microcontroller:** Arduino Uno
- **Input Sensor:** TMP36 or Potentiometer
- **Visual Output:** LCD1602
- **Status Light:** RGB LED
- **Audible Alert:** Piezo Buzzer
- **Passive Parts:** Resistors and Breadboard

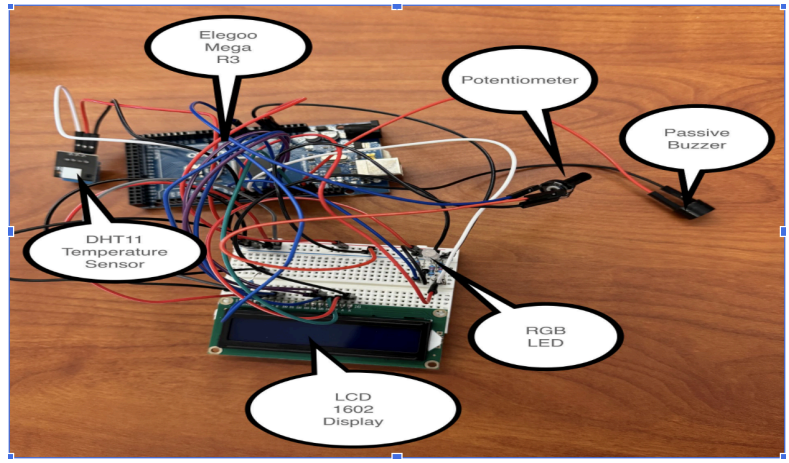
4. Physical Hardware Implementation

Moving to the lab bench required transitioning from "perfect" simulation to handling real-world component stability.

- **Wiring Layout:** Centralized on a breadboard to manage power rails for the Elegoo Mega R3.
- **Pin Assignments:** DHT11 (Digital Pin 1), Potentiometer (Analog A0), LCD (7, 8, 9, 10, 11, 12), RGB LED (4, 5, 6), and Buzzer (13) .
- **Physical Testing:** An ice bag test confirmed real-time transition to the "Cold" (Blue) state, while a body heat test triggered the 80°F alarm .

Key Changes from Simulation: We replaced the TMP36 with a physical DHT11 sensor, requiring the `DHT.h` digital library. We added a 10k potentiometer for LCD contrast and introduced a 1-second `setup()` delay to allow the DHT11 to stabilize . Additionally, the calibration potentiometer on A0 was added to "nudge" readings by ±10°F to account for sensor variance.





5. Challenges & Debugging

- **Sensor Communication:** The system initially reported a "Sensor error" because the physical DHT11 requires the `<DHT.h>` digital library, unlike the analog TMP36. We resolved this by integrating the library and adding a one-second delay in `setup()` for hardware stabilization.

- **Display Clarity:** The LCD appeared blank upon power-up. We diagnosed this as a contrast issue and resolved it by adding a dedicated 10k potentiometer to the contrast pin to manually dial in text focus.

- **Passive Buzzer Logic:** The alarm initially remained silent during high-temperature events. We learned that real buzzers require specific `tone()` frequencies,so we implemented a 1500Hz tone and a corresponding `noTone()` command to end the alert.

6. Code Architecture

The software is designed to be modular and reactive, using a "Read-Process-Update" loop.

- **Libraries:** We utilized `LiquidCrystal.h` for LCD commands and `DHT.h` to decode digital data from the DHT11 sensor.
- **Role of setup():** This function initializes the LCD, sets pin modes for the LED and Buzzer, and starts the DHT11 sensor after a stability delay .
- **Role of loop():** The loop continuously polls the sensors every second to update the display and monitor the alarm status .
- **Calibration Algorithm:** Using `analogRead()` on the A0 potentiometer, the code maps the 0–1023 input to a **plus or minus ten degree Fahrenheit** range, allowing for manual on-the-fly offset adjustments .
- **Core Logic:** The `determineColor()` function acts as a filter, assigning **Blue** (<67°F), **Green** (Normal), or **Red** (>72°F) to the RGB LED based on the adjusted temperature . If the value exceeds **80°F**, the system triggers the alarm state by clearing the LCD and sounding the 1500Hz tone .

7. Design Modifications

The project scope increased significantly during the transition from simulation to the laboratory environment. While the core goal remained constant, the physical build required several hardware and logic adjustments to ensure stability:

- **The Sensor Swap:** We replaced the simulated TMP36 with a physical DHT11, shifting from analog voltage readings to a digital protocol using the DHT.h library.

- **Manual Calibration (The A0 Knob):** We added a 10k potentiometer to Analog Pin A0 to "nudge" temperature readings by ±10°F, allowing us to test the 80°F alarm threshold without needing an external heat source.
- **Physical Visibility:** We added a second 10k potentiometer to manage the LCD contrast, a hardware necessity not modeled in the virtual version.
- **Stability Fixes:** A 1-second delay was added to setup() to allow the physical sensor time to stabilize after receiving power.

8. Reflection & Lessons Learned

The multi-tier alert system—combining the LCD, RGB LED, and buzzer—was a major success in providing clear safety feedback.

- **What Worked Well:** The A0 potentiometer was a critical addition, allowing us to validate the "Too Hot!!" logic instantly.
- **Lessons Learned:** We discovered that real components like LCD contrast and buzzer frequencies require manual tuning and specific code functions (like tone()) that are not always obvious in a simulation.
- **Surprises:** We were surprised by the high sensitivity of the DHT11 during the physical ice bag test; seeing the LED snap to blue in real-time proved our logic was solid .
- **Simulation Gaps:** Tinkercad failed to model environmental noise and hardware lag. In the physical world, we had to account for sensor "breathing" and inaccuracies through stability delays and calibration logic.

Appendix A: Full C++ Source Code (Sketch)

```
/*
 * Temperature Monitoring and Alert System
 * ---------------------------------------
 * Reads temperature from a DHT11 sensor and a manual calibration
 * potentiometer. Displays the adjusted value on a 16x2 LCD,
 * changes an RGB LED color based on range, and activates a
 * buzzer alarm if the temperature exceeds 80°F.
 */

#include <LiquidCrystal.h>
#include <DHT.h>

// ----- Hardware Pin Definitions -----
#define DHTPIN 1         // DHT11 sensor on digital pin 1
#define DHTTYPE DHT11    // Define sensor type
#define BUZZERPIN 13     // Passive buzzer on pin 13
const int potpin = A0;   // Calibration potentiometer on A0
```

```
// ----- LCD Pin Mapping (RS, E, D4, D5, D6, D7) -----

LiquidCrystal lcd(7, 8, 9, 10, 11, 12);



// ----- RGB LED Pins -----

const int RED_PIN   = 4;

const int GREEN_PIN = 5;

const int BLUE_PIN  = 6;



// Initialize DHT sensor

DHT dht(DHTPIN, DHTTYPE);



// RGB color arrays (PWM values)

static int RED[3]   = {255, 0, 0};   // Hot

static int GREEN[3] = {0, 255, 0};   // Normal

static int BLUE[3]  = {0, 0, 255};   // Cold
```

```
// ---------------------------------------------------

// determineColor()

// Returns pointer to RGB array based on temperature

// ---------------------------------------------------

int* determineColor(float temp) {

  if (temp < 67) {

    return BLUE;

  }

  else if (temp > 72) {

    return RED;

  }

  else {

    return GREEN;

  }

}



// ---------------------------------------------------
```

```
// colorWrite()

// Sets RGB LED color using PWM values

// ------------------------------------------------

void colorWrite(int RGB[3]) {

  analogWrite(RED_PIN,   RGB[0]);

  analogWrite(GREEN_PIN, RGB[1]);

  analogWrite(BLUE_PIN,  RGB[2]);

}



void setup() {

  lcd.begin(16, 2);

  lcd.print("Temp sensor kit!");



  pinMode(BUZZERPIN, OUTPUT);

  pinMode(RED_PIN, OUTPUT);

  pinMode(GREEN_PIN, OUTPUT);

  pinMode(BLUE_PIN, OUTPUT);
```

```
  delay(1000); // Stability delay for hardware

  dht.begin();

}



void loop() {

  // Read base temperature in Fahrenheit

  float f = dht.readTemperature(true);



  // Handle sensor failure

  if (isnan(f)) {

    lcd.clear();

    lcd.print("Sensor error");

    noTone(BUZZERPIN);

    return;

  }
```

```
// Manual Potentiometer Adjustment

int potValue = analogRead(potpin);

float adjustment = map(potValue, 0, 1023, -10, 10);

f = f + adjustment; // Apply user offset



// Update LED color

int* color = determineColor(f);

colorWrite(color);



// Alarm logic

if (f > 80) {

  lcd.clear();

  lcd.setCursor(0, 0);

  lcd.print("Too Hot!!");

  lcd.setCursor(0, 1);

  lcd.print(f);

  tone(BUZZERPIN, 1500); // 1500Hz Alarm tone
```

```
  }
  else {
    lcd.clear();
    lcd.print("Temp (F):");
    lcd.setCursor(0, 1);
    lcd.print(f);
    noTone(BUZZERPIN); // Silence alarm
  }


  delay(1000); // Delay for sensor stability
}
```

Appendix B: Tinkercad Link

- **Name of Design**: group1_circuit Temperature Monitoring System
- **Direct URL**: https://www.tinkercad.com/things/eJd4XoIyr0V/editel?sharecode=YALNnE9Zat3yzKr6kejnaYFviZ4L0PgX-wVaHa_LaEA